

СОДЕРЖАНИЕ

1 Разработка HLS-компонента lab1_z1	2
1.1 Цель и постановка задачи	2
1.2 Разработка исходных файлов	3
1.3 Директива PIPELINE OFF	5
1.4 C-моделирование и отладка.....	6
1.5 C-синтез и анализ результатов	8
1.6 Анализ Schedule Viewer	9
1.7 C/RTL co-simulation и временные диаграммы	10
1.8 Упаковка компонента в Vivado IP	12
1.9 Implementation и анализ результатов	13
1.10 Вывод.....	16
2 Разработка HLS-компонента lab1_z2.....	17
2.1 Цель и постановка задачи	17
2.2 Разработка исходных файлов	17
2.3 C-моделирование и отладка.....	19
2.4 C-синтез и анализ результатов	20
2.5 Анализ Schedule Viewer	21
2.6 C/RTL co-simulation и временные диаграммы	21
2.7 Вывод.....	23

1 Разработка HLS-компонента lab1_z1

1.1 Цель и постановка задачи

Целью работы является освоение полного маршрута проектирования HLS-компонента в среде AMD Vitis HLS: от описания алгоритма на языке C++ и разработки автоматического теста до C-синтеза, C/RTL co-simulation, упаковки результата в Vivado IP и выполнения этапа Implementation.

В рамках задания реализуется вычисление

$$outArr[i] = inArr[i] \cdot inA + inB + inC, \quad i = 0, 1, \dots, ROWS - 1.$$

Базовое значение ROWS равно 3. Для входного массива и скалярных входов используется тип short, для выходного массива — int. Тестовая программа должна не менее $N = 3$ раз формировать псевдослучайные входные данные, вычислять эталонный результат и автоматически сравнивать его с результатом тестируемой функции.

Параметры HLS-компонента приведены в таблице 1.

Параметр	Значение
Имя компонента	lab1_z1
Целевая микросхема	XC7Z020CLG484-1
Период тактового сигнала	10 нс
Clock uncertainty / margin	12 %
Flow target	Vivado IP
ROWS	3
N	3

1.2 Разработка исходных файлов

Для компонента разработаны три исходных файла: заголовочный файл lab1_z1.h, файл синтезируемой функции lab1_z1.cpp и тестовое окружение lab1_z1_test.cpp.

Определения типов данных, констант ROWS и N, а также прототип функции приведены на рисунке 1.

```
lab1_z1 > h lab1_z1.h > ...
1  #ifndef LAB1_Z1_H
2  #define LAB1_Z1_H
3
4  #define ROWS 3
5  #define N 3
6
7  typedef short din_type;
8  typedef int dout_type;
9
10 void lab1_z1(
11     din_type inArr[ROWS],
12     din_type inA,
13     din_type inB,
14     din_type inC,
15     dout_type outArr[ROWS]
16 );
17
18 #endif
```

Рисунок 1 — Заголовочный файл lab1_z1.h

Реализация синтезируемой функции представлена на рисунке 2.

```

lab1_z1 > C++ lab1_z1.cpp > ...
1  #include "lab1_z1.h"
2
3  void lab1_z1(
4      din_type inArr[ROWS],
5      din_type inA,
6      din_type inB,
7      din_type inC,
8      dout_type outArr[ROWS]
9  ) {
10     internal_loop:
11         for (int i = 0; i < ROWS; ++i) {
12             #pragma HLS PIPELINE off
13
14             din_type x = inArr[i];
15
16             dout_type mult = x * inA;
17             dout_type sumBC = inB + inC;
18
19             outArr[i] = mult + sumBC;
20         }
21     }

```

Рисунок 2 — Исходный код функции lab1_z1

В функции цикл `internal_loop` последовательно обрабатывает три элемента массива. На каждой итерации считывается очередной элемент `inArr[i]`, выполняется умножение на `inA`, вычисляется сумма `inB + inC` и формируется выходное значение `outArr[i]`.

Тестовое окружение приведено на рисунке 3.

```

// Генерация входных данных
inA = std::rand() % 1000;
inB = std::rand() % 1000;
inC = std::rand() % 1000;

for (int i = 0; i < ROWS; ++i) {
    inArr[i] = std::rand() % 1000;
}

// Вычисляем эталонный результат
for (int i = 0; i < ROWS; ++i) {
    expectedArr[i] =
        static_cast<dout_type>(inArr[i]) * inA
        + inB
        + inC;
}

// Запускаем тестируемую функцию
lab1_z1(inArr, inA, inB, inC, actualArr);

// Сравниваем результат
for (int i = 0; i < ROWS; ++i) {
    if (expectedArr[i] != actualArr[i]) {
        ++err_cnt;

        std::cout
            << "ERROR: index = " << i
            << ", expected = " << expectedArr[i]
            << ", actual = " << actualArr[i]
            << '\n';
    }
}

std::cout << "Simulation iteration " << iter << " finished\n";
}

std::cout << "Total error count: " << err_cnt << '\n';

if (err_cnt) {
    std::cout << "TEST FAILED\n";
} else {
    std::cout << "TEST PASSED\n";
}

return err_cnt;
}

```

Рисунок 3 — Исходный код тестового окружения lab1_z1_test.cpp

Для каждого запуска тест формирует псевдослучайные значения, отдельно вычисляет массив `expectedArr`, вызывает функцию `lab1_z1` и сравнивает `expectedArr` с `actualArr`. При несовпадении увеличивается счетчик `err_cnt`. Возврат нулевого значения соответствует успешному прохождению теста.

1.3 Директива PIPELINE OFF

Для цикла `internal_loop` задана директива HLS PIPELINE off. Она запрещает конвейеризацию цикла, поэтому следующая итерация не запускается как новая конвейерная итерация до завершения текущей. Это

позволяет явно наблюдать задержку каждой итерации в результатах планирования.

Поле HLS DIRECTIVES с установленной директивой показано на рисунке 4.



Рисунок 4 — Директива PIPELINE OFF для цикла internal_loop

1.4 С-моделирование и отладка

После разработки исходных файлов выполнено С-моделирование. Тест был запущен три раза с различными псевдослучайными входными значениями. Ошибок сравнения не обнаружено.

Результат успешного С-моделирования показан на рисунке 5.

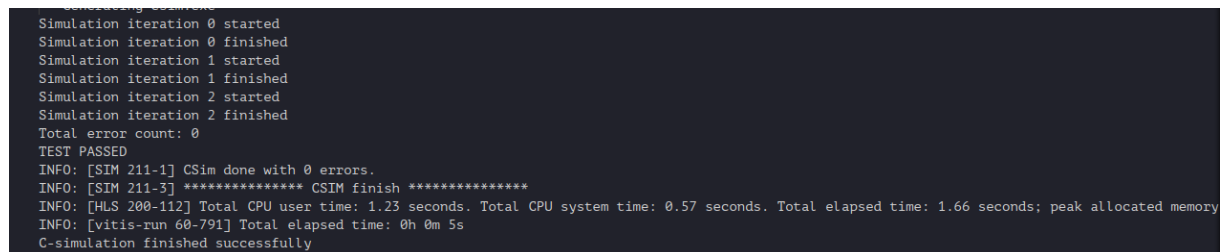


Рисунок 5 — Успешное прохождение C Simulation

Для проверки корректности самого тестового окружения в вычисление результата была намеренно внесена ошибка: к выходному значению добавлена единица. Измененный фрагмент функции приведен на рисунке 6.

```

#include "lab1_z1.h"

void lab1_z1(
    din_type inArr[ROWS],
    din_type inA,
    din_type inB,
    din_type inC,
    dout_type outArr[ROWS]
) {
    internal_loop:
    for (int i = 0; i < ROWS; ++i) {
        #pragma HLS PIPELINE off

        din_type x = inArr[i];

        dout_type mult = x * inA;
        dout_type sumBC = inB + inC;

        outArr[i] = mult + sumBC + 1;
    }
}

```

Рисунок 6 — Намеренное внесение ошибки в вычисляемый результат

При повторном запуске тест обнаружил несовпадения между ожидаемыми и фактическими значениями. Результат проверки представлен на рисунке 7.

```

0 ERROR: index = 1, expected = 140073, actual = 140074
1 ERROR: index = 2, expected = 11563, actual = 11564
2 Simulation iteration 1 finished
3 Simulation iteration 2 started
4 ERROR: index = 0, expected = 639762, actual = 639763
5 ERROR: index = 1, expected = 373422, actual = 373423
6 ERROR: index = 2, expected = 294762, actual = 294763
7 Simulation iteration 2 finished
8 Total error count: 9
9 TEST FAILED
10 @E Simulation failed: Function 'main' returns nonzero value '9'.
11 ERROR: [SIM 211-100] 'csim_design' failed: nonzero return value.
12 INFO: [SIM 211-3] ***** CSIM finish *****
13 ERROR: CSIM Failed
14 INFO: [HLS 200-112] Total CPU user time: 1.23 seconds. Total CPU system time: 0.54 seconds. Total elapsed time: 1.64 seconds; peak allocated memory
15 Failed to run c-simulation
16

```

Рисунок 7 — Обнаружение намеренно внесенной ошибки тестовым окружением

Дополнительно проверка выполнена в режиме Debug. На первой итерации ожидаемое значение равно 352108, фактическое — 352109, а счетчик `err_cnt` увеличен до 1. Это подтверждает, что тест корректно фиксирует ошибку.

Состояние переменных в режиме Debug показано на рисунке 8.

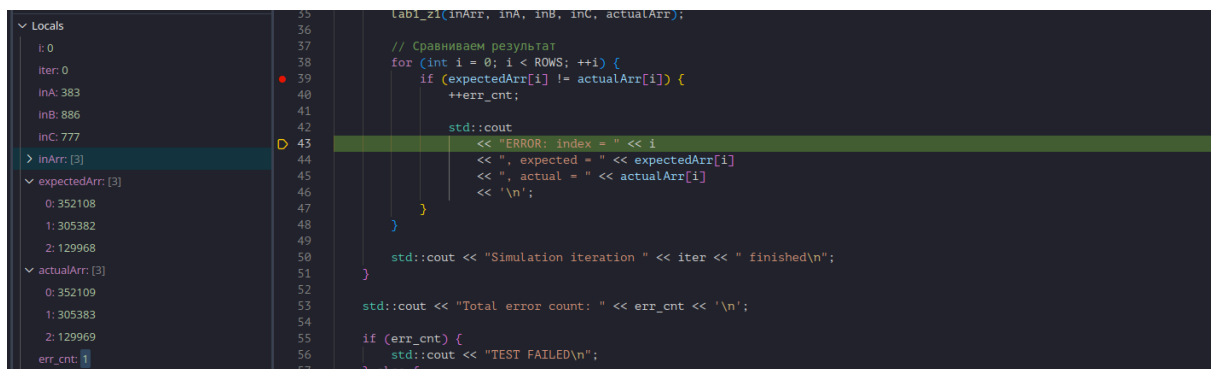


Рисунок 8 — Проверка обнаружения ошибки в режиме Debug

После проверки намеренное изменение было удалено, и повторное С-моделирование завершилось со статусом TEST PASSED.

1.5 С-синтез и анализ результатов

После успешного С-моделирования выполнен C Synthesis. Полученные оценки временных параметров и аппаратных затрат приведены на рисунке 9.

Estimated Quality of Results

Timing Estimate

TARGET	WITH UNCERTAINTY	ESTIMATED
10.00 ns	8.80 ns	3.153 ns

Performance & Resource Estimates

Рисунок 9 — Оценки временных параметров и аппаратных затрат после C Synthesis

Параметр	Полученное значение
Target clock period	10,00 нс
Clock period with uncertainty	8,80 нс
Estimated clock period	3,153 нс
Latency функции lab1_z1	160 нс (16 тактов)
Interval функции	17 тактов
Latency цикла internal_loop	150 нс (15 тактов)
Iteration Latency	5 тактов
Trip Count	3

При периоде 10 нс три последовательные итерации по 5 тактов дают 15 тактов, то есть 150 нс задержки цикла. Полная задержка функции составляет 16 тактов, или 160 нс. Интервал между последовательными запусками функции составляет 17 тактов.

Ресурс	Оценка HLS
BRAM	0
DSP	1
FF	74
LUT	90
URAM	0

В данной реализации операция умножения была отображена HLS на один DSP-блок. Небольшие массивы не привели к использованию BRAM или URAM. Оцененный период 3,153 нс существенно меньше целевого периода 10 нс, поэтому предварительная временная оценка удовлетворяет ограничению.

1.6 Анализ Schedule Viewer

Планировщик позволяет сопоставить операции исходного алгоритма с тактами аппаратного выполнения. Результат Schedule Viewer приведен на рисунке 10.

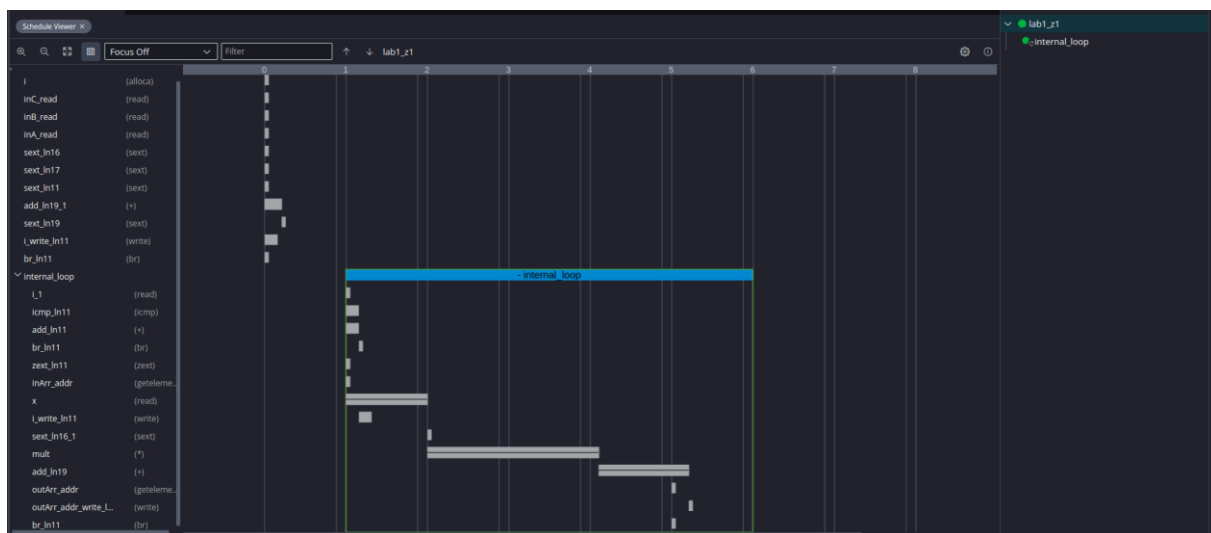


Рисунок 10 — Расписание операций в Schedule Viewer

В начале функции считываются скалярные входы inA, inB и inC. Сложение $\text{inB} + \text{inC}$ вынесено HLS за пределы цикла, поскольку эти значения не изменяются между итерациями. Внутри `internal_loop` последовательно выполняются чтение `inArr[i]`, умножение $\text{inArr}[i] \cdot \text{inA}$, итоговое сложение и запись результата в `outArr[i]`.

Задержка одной итерации равна 5 тактам. При трех итерациях и отключенной конвейеризации суммарная задержка цикла равна 15 тактам. Полученное расписание согласуется с оценкой C Synthesis.

1.7 C/RTL co-simulation и временные диаграммы

На этапе C/RTL co-simulation проверено соответствие поведения синтезированного RTL исходной C-модели. Тест завершился со статусом PASS. Timeline Trace одновременно показывает три запуска функции и измеренные значения Π и Latency.

Результат co-simulation и Timeline Trace приведены на рисунке 11.

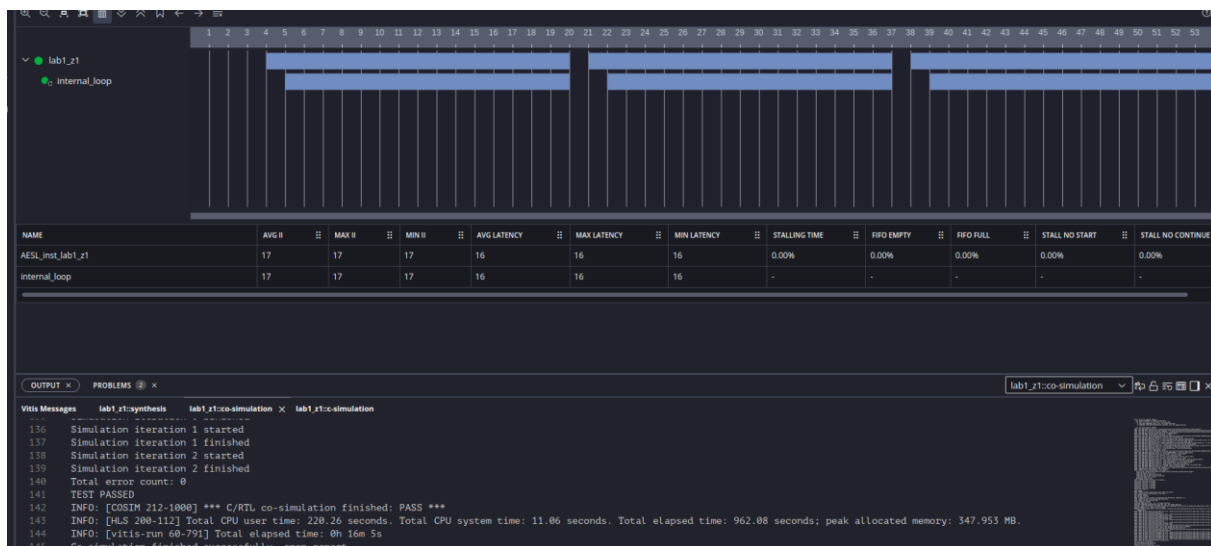


Рисунок 11 — Результат C/RTL co-simulation и Timeline Trace

Параметр	C Synthesis	C/RTL co-simulation
Latency функции	16 тактов	16 тактов
Interval / II	17 тактов	17 тактов
Stalling Time	—	0,00 %

Результаты co-simulation совпадают с предварительными оценками C Synthesis: Latency функции равна 16 тактам, а Interval — 17 тактам. При периоде 10 нс это соответствует 160 нс и 170 нс соответственно.

Для детального анализа использован Wave Viewer. На временной диаграмме видны сигналы управления `ap_start`, `ap_ready` и `ap_done`, скалярные входы, а также сигналы адресации, чтения и записи массивов.

Фрагмент временной диаграммы первого запуска показан на рисунке 12.

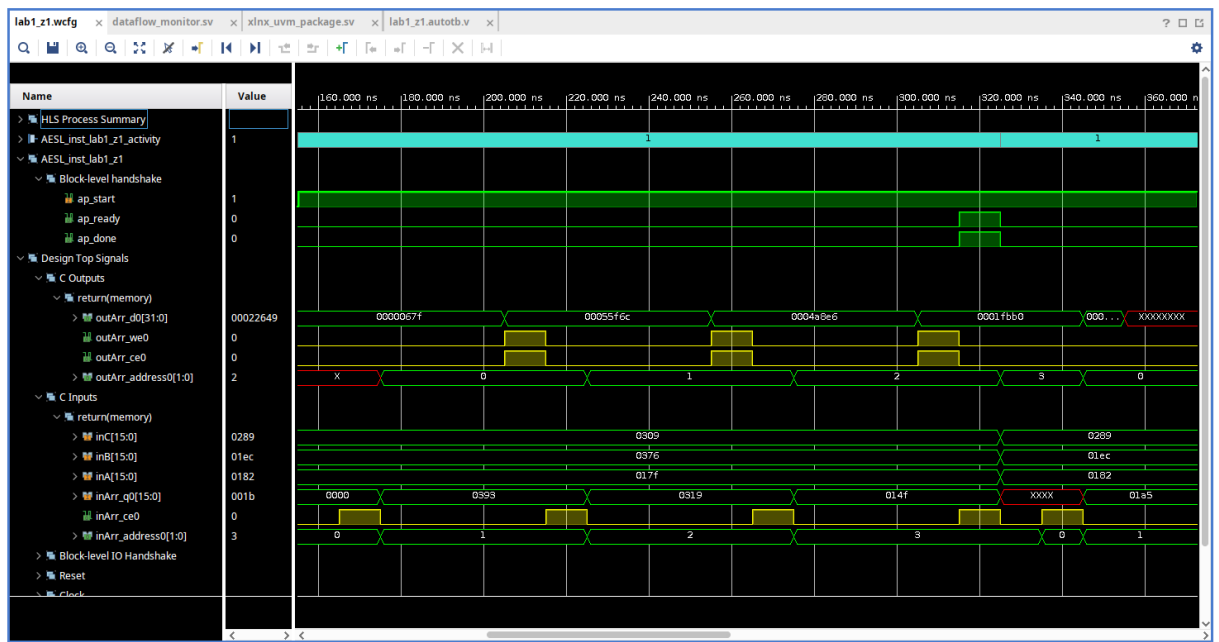


Рисунок 12 — Временная диаграмма RTL-модели в Wave Viewer

Сигнал `inArr_ce0` совместно с `inArr_address0` определяет моменты чтения элементов входного массива. Сигналы `outArr_ce0` и `outArr_we0` совместно с `outArr_address0` определяют моменты записи результатов. Адреса 0, 1 и 2 соответствуют трем итерациям цикла.

По временной диаграмме подтверждаются следующие значения: Iteration Latency = 5 тактов = 50 нс, loop latency = 15 тактов = 150 нс, Latency функции = 16 тактов = 160 нс, Interval функции = 17 тактов = 170 нс.

1.8 Упаковка компонента в Vivado IP

Синтезированное RTL-описание упаковано в формат Vivado IP Catalog. В процессе упаковки сформированы описание компонента `component.xml` и архив IP.

Сообщения об успешном завершении этапа Package приведены на рисунке 13.

```
INFO: Calling post_process_vitis to specialize IP
INFO: Calling post_process_sysgen to specialize IP
Generating sysgen info xml from json file
INFO: Created IP /home/zxcuser1/Dev/vitis/projects/lab1/lab1_z1/lab1_z1/hls/impl/ip/component.xml
INFO: Created IP archive /home/zxcuser1/Dev/vitis/projects/lab1/lab1_z1/lab1_z1/hls/impl/ip/xilinx_com_hls_lab1_z1_1_0.zip
INFO: [Common 17-206] Exiting Vivado at Sun Sep 27 16:15:01 2026...
INFO: [HLS 200-802] Generated output file lab1_z1/lab1_z1.zip
INFO: [HLS 200-112] Total CPU user time: 12.76 seconds. Total CPU system time: 1.36 seconds. Total elapsed time: 6.55 seconds; peak allocated memoz
INFO: [vitis-run 60-791] Total elapsed time: 0h 0m 10s
Package finished successfully
```

Рисунок 13 — Успешное завершение упаковки HLS-компонента в IP

Содержимое каталога сформированного IP показано на рисунке 14.

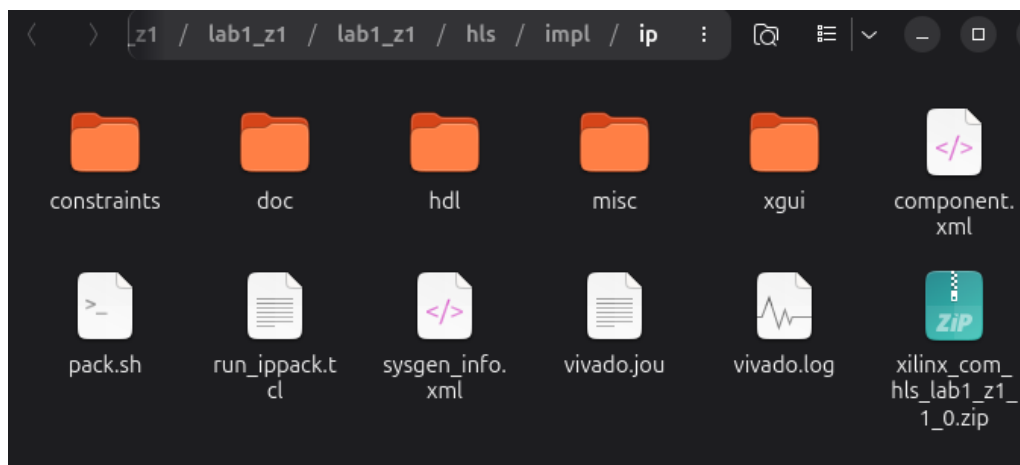


Рисунок 14 — Каталог сформированного Vivado IP

Созданный архив имеет имя `xilinx_com_hls_lab1_z1_1_0.zip` и расположен в каталоге `hls/impl/ip`.

1.9 Implementation и анализ результатов

На заключительном этапе выполнена реализация компонента средствами Vivado, включающая синтез, размещение и трассировку. Все этапы завершились без ошибок, маршрутизация выполнена полностью.

Сообщение об успешном завершении Implementation приведено на рисунке 15.

```

TIMESTAMP: HLS-REPORT: implementation end: 2026-09-27 16:18:25 MSK
INFO: HLS-REPORT: impl run complete: worst setup slack (WNS)=7.962470, worst hold slack (WHS)=0.209728, total pulse width slack(TPWS)=0.000000, nu
# hls_vivado_reports_finalize $report_options
TIMESTAMP: HLS-REPORT: all reports complete: 2026-09-27 16:18:25 MSK
INFO: [Common 17-206] Exiting Vivado at Sun Sep 27 16:18:25 2026...
INFO: [HLS 200-802] Generated output file lab1_z1/lab1_z1.zip
INFO: [HLS 200-112] Total CPU user time: 142.38 seconds. Total CPU system time: 13.06 seconds. Total elapsed time: 108.69 seconds; peak allocated
INFO: [vitis-run 60-791] Total elapsed time: 0h 1m 52s
Implementation finished successfully, open_report.

```

Рисунок 15 — Успешное завершение этапа Implementation

После реализации получены окончательные оценки временных параметров. Целевой период составлял 10 нс, достигнутый период после синтеза и после Implementation — 2,154 нс. Требование по времени выполнено.

Финальные временные параметры приведены на рисунке 16.

```
#=== Final timing ===
CP required:                10.000
CP achieved post-synthesis:  2.154
CP achieved post-implementation: 2.154
Timing met
```

Рисунок 16 — Финальные временные параметры после Implementation

Аппаратные затраты после физической реализации приведены на рисунке 17.

```
#=== Post-Implementation Resource usage ===
SLICE:      8
LUT:        23
FF:         10
DSP:         1
BRAM:        0
URAM:        0
LATCH:       0
SRL:         0
CLB:         0
```

Рисунок 17 — Использование ресурсов после Implementation

Ресурс	Оценка HLS	После Implementation
LUT	90	23
FF	74	10
DSP	1	1
BRAM	0	0
URAM	0	0

После реализации количество LUT и FF уменьшилось относительно предварительной HLS-оценки, тогда как число DSP осталось равным единице. Это связано с дополнительной логической и физической оптимизацией, выполняемой Vivado для конкретной целевой микросхемы.

Анализ Fail Fast не выявил потенциальных проблем. Все проверяемые критерии имеют статус ОК; использование ресурсов находится значительно ниже пороговых значений.

Результат анализа Fail Fast приведен на рисунке 18.

Design Summary			
impl_1			
xc7z020clg484-1			
Criteria	Guideline	Actual	Status
LUT	70%	0.04%	OK
FD	50%	0.01%	OK
LUTRAM+SRL	25%	0.00%	OK
MUXF7	15%	0.00%	OK
DSP	80%	0.45%	OK
RAMB/FIFO	80%	0.00%	OK
DSP+RAMB+URAM (Avg)	70%	0.45%	OK
BUFGCE* + BUFGCTRL	24	0	OK
DONT_TOUCH (cells/nets)	0	0	OK
MARK_DEBUG (nets)	0	0	OK
Control Sets	998	2	OK
Average Fanout for modules > 100k cells	4	0.88	OK
Max Average Fanout for modules > 100k cells	4	0	OK
Non-FD high fanout nets > 10k loads	0	0	OK
TIMING-6 (No common primary clock between related clocks)	0	0	OK
TIMING-7 (No common node between related clocks)	0	0	OK
TIMING-8 (No common period between related clocks)	0	0	OK
TIMING-14 (LUT on the clock tree)	0	0	OK
TIMING-35 (No common node in paths with the same clock)	0	0	OK
Number of paths above max LUT budgeting (0.575ns)	0	0	OK
Number of paths above max Net budgeting (0.403ns)	0	0	OK

Рисунок 18 — Результаты анализа Fail Fast после Implementation

Анализ временных путей также не выявил нарушений. Итоговое значение worst setup slack (WNS) составляет 7,962470 нс, worst hold slack (WHS) — 0,209728 нс; число неразведенных соединений равно нулю. Положительные значения WNS и WHS подтверждают выполнение требований setup- и hold-времени.

Предупреждения, возникающие при маршрутизации, относятся к режиму Out-of-Context: для внешних портов IP не заданы окончательные физические расположения выводов, а для ap_clk не определено положение источника тактового сигнала верхнего уровня. Эти предупреждения не привели к ошибкам или критическим предупреждениям и не помешали успешному завершению Implementation.

1.10 Вывод

В ходе выполнения задания lab1_z1 разработан и проверен HLS-компонент, вычисляющий выражение $\text{outArr}[i] = \text{inArr}[i] \cdot \text{inA} + \text{inB} + \text{inC}$. Создано автоматическое тестовое окружение, подтверждена его способность обнаруживать ошибки, выполнены C-моделирование и отладка.

После C-синтеза проведен анализ расписания операций, задержек и аппаратных затрат. При отключенной конвейеризации задержка одной итерации цикла составила 5 тактов, задержка цикла — 15 тактов, задержка функции — 16 тактов, а интервал между запусками — 17 тактов. Эти значения подтверждены C/RTL co-simulation и анализом временных диаграмм.

Компонент успешно упакован в Vivado IP и реализован для микросхемы XC7Z020CLG484-1. После Implementation использовано 23 LUT, 10 FF и 1 DSP, временные ограничения выполнены, а Fail Fast не выявил потенциальных проблем.

2 Разработка HLS-компонента lab1_z2

2.1 Цель и постановка задачи

Целью второй части лабораторной работы является исследование синтеза простой комбинационной функции в среде AMD Vitis HLS и сопоставление исходного C++-описания с полученной аппаратной реализацией.

В рамках задания реализуется вычисление

$$\text{sum} = \text{inA} + \text{inB} + \text{inC}.$$

Для входных данных inA, inB и inC используется тип short, для результата sum — тип int. Тестовая программа должна не менее $N = 3$ раз формировать псевдослучайные входные данные, вычислять эталонный результат и автоматически сравнивать его с результатом тестируемой функции.

Параметры HLS-компонента приведены в таблице 6.

Параметр	Значение
Имя компонента	lab1_z2
Целевая микросхема	XC7Z020CLG484-1
Период тактового сигнала	50 нс
Clock uncertainty / margin	10 %
Flow target	Vivado IP
N	3

2.2 Разработка исходных файлов

Для компонента разработаны три исходных файла: заголовочный файл lab1_z2.h, файл синтезируемой функции lab1_z2.cpp и тестовое окружение lab1_z2_test.cpp.

Определения типов данных, значения N и прототип функции приведены на рисунке 19.

```

lab1_z2 > h lab1_z2.h > ...
1  #ifndef LAB1_Z2_H
2  #define LAB1_Z2_H
3
4  typedef short din_type;
5  typedef int dout_type;
6
7  #define N 3
8
9  dout_type lab1_z2(din_type inA, din_type inB, din_type inC);
10 #endif

```

Рисунок 19 — Заголовочный файл lab1_z2.h

Реализация синтезируемой функции представлена на рисунке 20. Функция возвращает сумму трех входных значений.

```

lab1_z2 > g++ lab1_z2.cpp > lab1_z2
1  #include "lab1_z2.h"
2
3  dout_type lab1_z2(din_type inA, din_type inB, din_type inC) {
4      return inA + inB + inC;
5  }

```

Рисунок 20 — Исходный код функции lab1_z2

Тестовое окружение приведено на рисунке 21. Для каждой итерации генерируются псевдослучайные значения inA, inB и inC, вычисляется эталонная сумма exp_sum, вызывается функция lab1_z2 и выполняется сравнение результатов. При несовпадении увеличивается счетчик err_cnt.

```

lab1_z2 > g++ lab1_z2_test.cpp > ...
1  #include "lab1_z2.h"
2  #include <cstdlib>
3  #include <iostream>
4
5  int main()
6  {
7      int err_cnt = 0;
8      for (int iter = 0; iter < N; ++iter)
9      {
10         std::cout << "iteration " << iter + 1 << " starting\n";
11         din_type inA = std::rand() % 1337, inB = std::rand() % 1337, inC = std::rand() % 1337;
12
13         dout_type exp_sum = inA + inB + inC;
14
15         dout_type sum = lab1_z2(inA, inB, inC);
16
17         if (sum != exp_sum)
18         {
19             err_cnt++;
20             std::cout << "expected: " << exp_sum << std::endl
21                 << "actual: " << sum << std::endl
22                 << "inA: " << inA << " inB: " << inB << " inC: " << inC << std::endl;
23         }
24
25         std::cout << "iteration " << iter + 1 << " finished\n";
26     }
27
28     if (!err_cnt)
29     {
30         std::cout << "TEST PASSED!" << std::endl;
31         return err_cnt;
32     }
33
34     std::cout << "TEST FAILED!" << std::endl;
35     return err_cnt;
36 }

```

Рисунок 21 — Исходный код тестового окружения lab1_z2_test.cpp

2.3 С-моделирование и отладка

После разработки исходных файлов выполнено С-моделирование. Тест выполнил три итерации, ошибок сравнения не обнаружено. Результат успешного моделирования показан на рисунке 22.

```
41      Compiling ../../../../lab1_z2.cpp in release mode
42      Generating csim.exe
43      iteration 1 starting
44      iteration 1 finished
45      iteration 2 starting
46      iteration 2 finished
47      iteration 3 starting
48      iteration 3 finished
49      TEST PASSED!
50      INFO: [SIM 211-1] CSim done with 0 errors.
51      INFO: [SIM 211-3] ***** CSIM finish *****
52      INFO: [HLS 200-112] Total CPU user time: 1.22 seconds. Total CPU system time: 0.56 seconds. Total elapsed time: 1.64 seconds; peak allocated memory
53      INFO: [vitis-run 60-791] Total elapsed time: 0h 0m 5s
54      C-simulation finished successfully
55
```

Рисунок 22 — Успешное прохождение C Simulation

Для проверки корректности тестового окружения в вычисление результата была намеренно внесена ошибка: из суммы вычиталась единица. Измененный фрагмент функции приведен на рисунке 23.

```
lab1_z2 > C++ lab1_z2.cpp > lab1_z2
1  #include "lab1_z2.h"
2
3  dout_type lab1_z2(din_type inA, din_type inB, din_type inC) {
4      return inA + inB + inC - 1;
5  }
```

Рисунок 23 — Намеренное внесение ошибки в вычисляемый результат

При повторном запуске тест обнаружил несовпадение для каждой из трех итераций. Например, для первого набора входных данных ожидаемое значение равно 2219, а фактическое — 2218. Результат проверки показан на рисунке 24.

```
iteration 1 starting
expected: 2219
actual: 2218
inA:1198 inB: 214 inC: 807
iteration 1 finished
iteration 2 starting
expected: 1563
actual: 1562
inA:1265 inB: 85 inC: 213
iteration 2 finished
iteration 3 starting
expected: 1287
actual: 1286
inA:465 inB: 93 inC: 729
iteration 3 finished
TEST FAILED!
@E Simulation failed: Function 'main' returns nonzero value '3'.
ERROR: [SIM 211-100] 'csim_design' failed: nonzero return value.
INFO: [SIM 211-3] ***** CSIM finish *****
ERROR: CSIM Failed
INFO: [HLS 200-112] Total CPU user time: 1.21 seconds. Total CPU system time: 0.57 seconds. Total elapsed time: 1.64 seconds; peak allocated memory
Failed to run c-simulation
```

Рисунок 24 — Обнаружение намеренно внесенной ошибки тестовым окружением

Дополнительно проверка выполнена в режиме Debug. На первой итерации значения `exp_sum` и `sum` равны 2219 и 2218 соответственно, а

счетчик `err_cnt` после обнаружения несовпадения равен 1. Состояние переменных показано на рисунке 25.

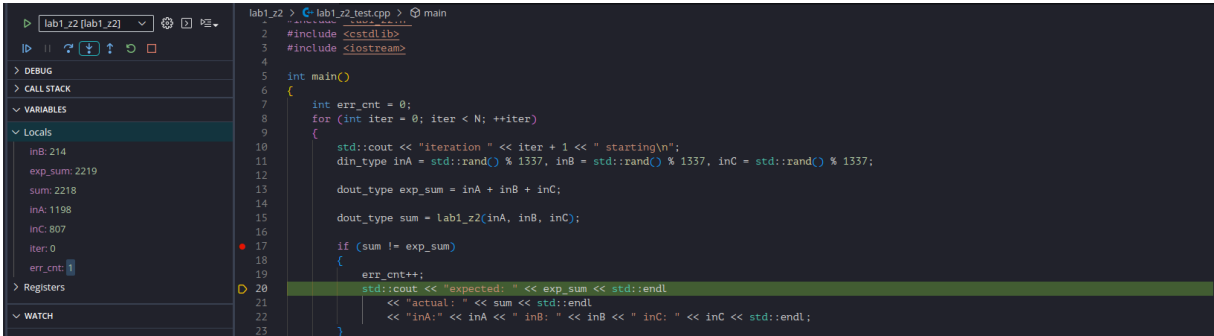


Рисунок 25 — Проверка обнаружения ошибки в режиме Debug

После проверки намеренное изменение было удалено, функция возвращена к исходному выражению `inA + inB + inC`, а повторное C-моделирование завершилось со статусом `TEST PASSED`.

2.4 C-синтез и анализ результатов

После успешного C-моделирования выполнен C Synthesis. Оценки временных параметров и аппаратных затрат приведены на рисунке 26.

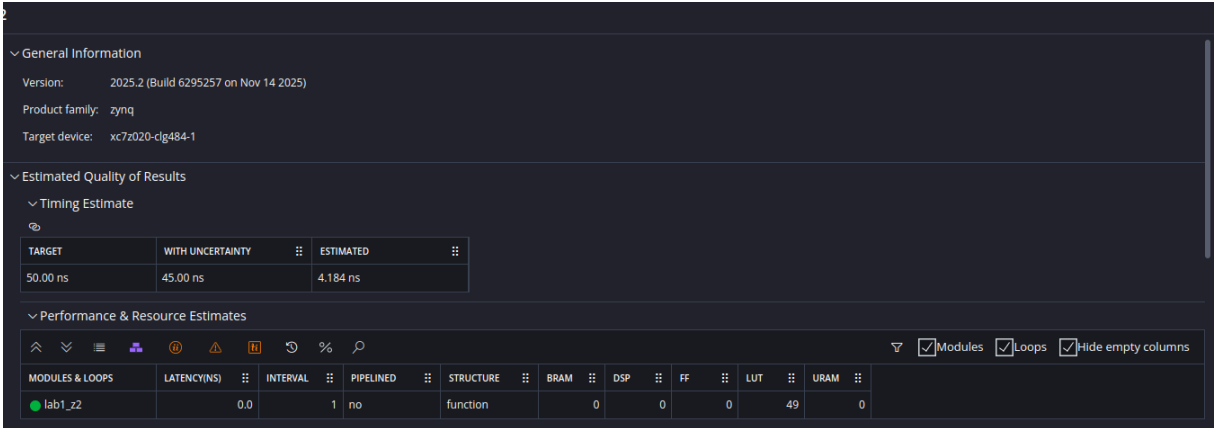


Рисунок 26 — Оценки временных параметров и аппаратных затрат после C Synthesis

Параметр	Полученное значение
Target clock period	50,00 нс
Clock period with uncertainty	45,00 нс
Estimated	4,184 нс
Latency	0,0 нс
Interval	1
Pipelined	no
Ресурс	Оценка HLS
BRAM	0
DSP	0
FF	0
LUT	49
URAM	0

Оценка задержки 4,184 нс существенно меньше доступных 45 нс с учетом 10%-го margin, поэтому временное ограничение выполняется с большим запасом. Для реализации функции используются 49 LUT. DSP и блоки памяти не требуются, поскольку алгоритм содержит только операции сложения и не использует массивы. Отсутствие FF и нулевая тактовая задержка соответствуют комбинационному характеру синтезированной схемы.

2.5 Анализ Schedule Viewer

Расписание операций, сформированное Vitis HLS, приведено на рисунке 27.

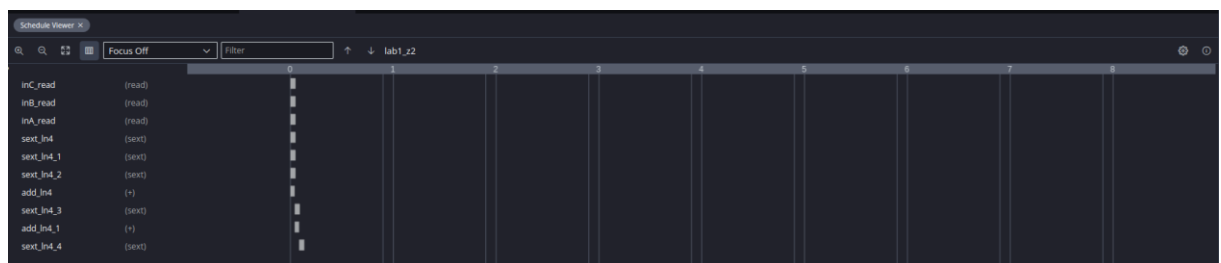


Рисунок 27 — Расписание операций в Schedule Viewer

В Schedule Viewer сначала отображается чтение входных значений inA, inB и inC. Операции sext соответствуют знаковому расширению входных значений типа short перед арифметикой с результатом типа int. Далее выполняются две операции сложения add_in4 и add_in4_1, реализующие выражение $inA + inB + inC$.

Все операции размещены внутри одного тактового интервала и не разделены регистрами. Это согласуется с результатами C Synthesis: аппаратная реализация является комбинационной и не имеет промежуточных FF.

2.6 C/RTL co-simulation и временные диаграммы

На этапе C/RTL co-simulation проверено соответствие поведения синтезированного RTL исходной C-модели. Все три тестовые итерации завершились успешно, а симулятор сообщил статус PASS. Результат приведен на рисунке 28.

```

start_gui
INFO: [Common 17-206] Exiting xsim at Sun Sep 27 18:44:34 2026...
INFO: [COSIM 212-316] Starting C post checking ...
iteration 1 starting
iteration 1 finished
iteration 2 starting
iteration 2 finished
iteration 3 starting
iteration 3 finished
TEST PASSED!
INFO: [COSIM 212-1000] *** C/RTL co-simulation finished: PASS ***
INFO: [COSIM 212-210] Design is translated to an combinational logic. II and Latency will be marked as all 0.
INFO: [HLS 200-112] Total CPU user time: 171 seconds. Total CPU system time: 8.4 seconds. Total elapsed time: 478.85 seconds; peak allocated memory
INFO: [vitis-run 60-791] Total elapsed time: 0h 8m 2s
Co-simulation finished successfully, open report.

```

Рисунок 28 — Успешное завершение C/RTL co-simulation

В отчёте C Synthesis для комбинационной функции указан Interval = 1, тогда как в отчёте C/RTL co-simulation значения II и Latency отображаются как 0, поскольку для комбинационной схемы отсутствуют тактовые транзакции. Для анализа интерфейсных сигналов использован Wave Viewer.

На рисунке 29 показаны сигналы управления ap_start, ap_ready, ap_done и ap_idle, а также сигнал сброса ap_rst и служебные сигналы тестового окружения.

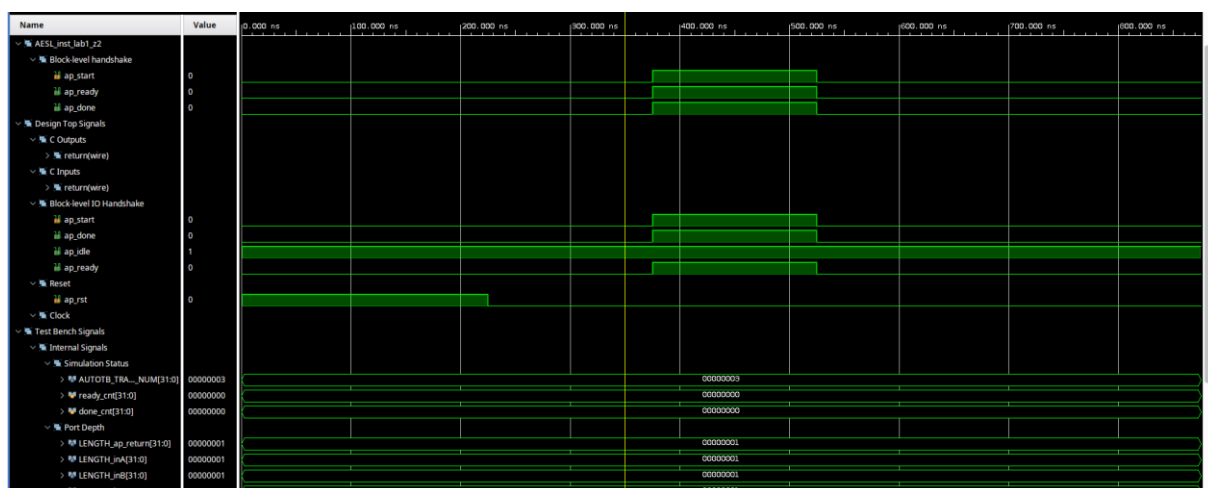


Рисунок 29 — Управляющие сигналы RTL-модели в Wave Viewer

На рисунке 30 показаны входы inA, inB, inC и выход ap_return. Для первого набора входных данных inA = 1198, inB = 214 и inC = 807, а ap_return = 2219, что соответствует сумме входов. Для второго и третьего наборов получены значения 1563 и 1287 соответственно; они также совпадают с ожидаемыми результатами.



Рисунок 30 — Входные и выходной сигналы RTL-модели в Wave Viewer

Временная диаграмма подтверждает результаты функционального теста и С-синтеза: выход формируется комбинационной логикой из трех входных операндов, без последовательной цепочки регистров между операциями.

2.7 Вывод

В ходе выполнения задания lab1_z2 разработан и проверен HLS-компонент, вычисляющий выражение $sum = inA + inB + inC$. Создано автоматическое тестовое окружение, подтверждена его способность обнаруживать намеренно внесенную ошибку, выполнены С-моделирование и отладка.

После С-синтеза установлено, что функция реализуется как простая комбинационная схема. Оценочная задержка составляет 4,184 нс при доступных 45 нс с учетом margin. Для реализации используется 49 LUT; DSP, BRAM, URAM и FF не используются. Schedule Viewer подтверждает чтение трёх входов и выполнение двух операций сложения в рамках одной комбинационной цепи без промежуточных регистров.

C/RTL co-simulation завершилась со статусом PASS. Анализ Wave Viewer подтвердил соответствие значений ap_return сумме входных сигналов для всех трех тестовых наборов.